MEMBER 4 • `UI` PACKAGE, SWING/GRAPHICS/IO LIBRARIES & LIVE DEMO

CricBid Review 2 Guide (From Scratch)

Aligned 100% with the Review 2 Marking Rubric: Packages & Interfaces (5M) + Java Libraries & Exception Handling (5M) + Presentation (5M).

✅ YOUR REVIEW 2 RUBRIC CHECKLIST (UI + LIBRARIES + LIVE 3-LAPTOP DEMO!)

📦 **Packages:** `com.cricbid.ui` (9 panels & dialogs)

🔌 **Interfaces:** `AuctionObserver`, `ActionListener`, `Consumer` / `BiConsumer`

📚 **Java Libraries:** `javax.swing.*`, `CardLayout`, `Graphics2D`, `PrintWriter`, `Collectors`, `NetworkInterface`

🛡️ **Exception Handling:** UI `try-catch` alerts for `AuthenticationException`, `IOException`, `SQLException`

1. Your Updated 2-Minute Speech (Rubric-Aligned)

"Finally, I'll explain our `com.cricbid.ui` package, the Swing, 2D graphics, and I/O libraries we used, UI-level exception handling, and our role-based multi-laptop upgrades.

In `com.cricbid.ui`, `MainFrame` uses `java.awt.CardLayout` to manage nine screens. For **interfaces**, `LiveAuctionPanel` implements `AuctionObserver`, while panels communicate via `ActionListener` and `java.util.function.Consumer` callbacks. For **Java libraries**, `ReportsPanel` uses `java.awt.Graphics2D` to render custom spending bar charts and role pie charts, and `ReportService` uses `java.util.stream.Collectors` and `java.io.PrintWriter` with `JFileChooser` to export CSV reports. Across the UI, **exception handling** catches `AuthenticationException`, `CricBidException`, and `IOException`—such as port conflicts or network drops—to display clean `JOptionPane` alerts instead of crashing.

For Review 2, first, we deferred all secondary panel DB queries until after login so remote laptops never query `localhost` at startup, and used `java.net.NetworkInterface` in `NetworkUtils` so `LoginPanel` auto-displays Laptop 1's Hotspot IP.

Second, we built strict role-based separation: Laptop 1 logs in as `admin` (Auctioneer Host) to configure franchises and control Start, Pause, and Next Player with no Place Bid button. Laptops 2 and 3 log in as `owner1` and `owner2` (Thunder Hawks and Iron Wolves), skipping Setup into `LiveAuctionPanel` with synced timers, `[You]` / `[Participant]` badges, and Host-only round advancement after `SoldPopupDialog`. Let's now demonstrate the live auction across our three laptops."

2. Basic Java Dictionary (Explained From Zero!)

Java Rubric Term	Simple Everyday Meaning
`extends JPanel implements AuctionObserver`	Shows class inheritance + interface implementation together! `extends JPanel` makes `LiveAuctionPanel` a Swing screen; `implements AuctionObserver` subscribes it to live auction updates.
`CardLayout` (`java.awt` library)	Stacks all 9 screens inside `MainFrame` like a deck of cards so switching screens swaps panels cleanly inside one window.

`Graphics2D` (`java.awt` library)	Java's 2D drawing engine used in `ReportsPanel` to custom-paint anti-aliased horizontal bar charts (`fillRoundRect`) and pie charts (`fillArc`).
`PrintWriter` & `Collectors` Libraries	`java.io.PrintWriter` writes sold player rows into `.csv` files for Excel; `java.util.stream.Collectors` filters and ranks top value picks.
`Consumer` / `BiConsumer` Interfaces	Functional interfaces from `java.util.function` (`user -> ...`) used by `LoginPanel` , `SetupPanel` , and `AuctionClient` to pass events cleanly back to `MainFrame` .

3. File-by-File Code Walkthrough (What to Open & Point At)

File 1: `MainFrame.java` — `CardLayout`, Deferred DB Queries & Port `IOException` Handling

📂 Open: src/main/java/com/cricbid/ui/MainFrame.java (Lines 286–380)

```
public void showCard(String cardName) {
    if (currentUser != null) {          // Deferred DB queries until AFTER login sets Host IP!
        switch (cardName) {
            case CARD_POOL:    playerPoolPanel.refreshPlayers(); break;
            case CARD_TEAMS:   teamsPanel.refreshTeams();        break;
            case CARD_REPORTS: reportsPanel.refreshReports();    break;
            case CARD_SUMMARY: summaryPanel.refreshSummary();    break;
        }
    }
    cardLayout.show(cardsPanel, cardName); // java.awt.CardLayout library!
}

// Inside onLoginSuccess (Lines 334–379): Admin Host vs Team Owner Participant
if (isAdmin) {
    this.auctionServer = new AuctionServer(serverPort, auctionService, teamService);
    try {
        this.auctionServer.start();
    } catch (IOException ex) {          // UI Exception Handling for Port Busy!
        JOptionPane.showMessageDialog(this,
            "Could not start Host Socket Server on TCP port " + serverPort + ":
" + ex.getMessage(),
            "Host Server Port Busy", JOptionPane.WARNING_MESSAGE);
    }
    showCard(CARD_SETUP);
} else {
    this.auctionClient = new AuctionClient(hostIp, serverPort, ...);
    this.auctionClient.start();
    showCard(CARD_LIVE);                // Participant skips Setup -> straight to Live Auction!
}
```

File 2: `LoginPanel.java` & `NetworkUtils.java` — `NetworkInterface` Library & Login Exceptions

📂 Open: src/main/java/com/cricbid/network/NetworkUtils.java (Lines 14–48) & ui/LoginPanel.java (Lines 180–198)

- **`java.net.NetworkInterface` Library:** `NetworkUtils.getLocalLanIp()` scans network interfaces inside `try-catch (SocketException)`, skipping `127.0.0.1` to auto-display Laptop 1's Hotspot IPv4 address on `LoginPanel`.
- **UI Exception Handling in `LoginPanel`:** Catches `AuthenticationException` (wrong password) and `SQLException` (cannot reach Host MySQL IP) and displays user-friendly `JOptionPane` dialogs.

File 3: `LiveAuctionPanel.java` — `AuctionObserver` Interface, Role Separation & Sync

📂 Open: src/main/java/com/cricbid/ui/LiveAuctionPanel.java (Lines 41, 105–135, 450–457, 502–508, 597–603)

```
public class LiveAuctionPanel extends JPanel implements AuctionObserver { // Inheritance +
Interface!
...
// 1. Role Separation: Admin gets Pause/Skip; Team Owners get Place Bid (Lines 113–116)
bidSpinner.setVisible(!isAdmin);
bidderBtnRow.setVisible(!isAdmin);
adminControlRow.setVisible(isAdmin);

// 2. Remote Bid with IOException Handling (Lines 450–457)
if (auctionClient != null) {
    try {
        auctionClient.sendBid(humanTeam.getTeamId(), bidAmount);
    } catch (IOException ex) {
        JOptionPane.showMessageDialog(this, ex.getMessage(), "Network Error",
JOptionPane.WARNING_MESSAGE);
    }
    return;
}

// 3. [You] vs [Participant] Team Badge (Lines 502–508)
if (humanTeam != null && team.getTeamId() == humanTeam.getTeamId()) return team.getName() + "
[You]";

// 4. Only Host advances round after SoldPopupDialog closes! (Lines 597–603)
if (auctionClient == null) {
    auctionService.presentNextPlayer();
}
```

File 4: `ReportsPanel.java` & `ReportService.java` — `Graphics2D`, `PrintWriter` & `Collectors`

📂 Open: src/main/java/com/cricbid/ui/ReportsPanel.java (Lines 155–276) & service/ReportService.java (Lines 70–107)

Exact Words to Say to Sir (Showcases 4 Java Libraries at Once!):

"Sir, in `ReportsPanel` we used `java.awt.Graphics2D` inside `paintComponent()` to draw custom anti-aliased spending bar charts and role pie charts. And when clicking Export CSV, `JFileChooser` passes the file to `ReportService`, which uses `java.util.stream.Collectors` for stream filtering and `java.io.PrintWriter` inside `try-with-resources` to write the `.csv` file while handling `IOException` and `SQLException`."

4. Viva Questions Sir Might Ask You (+ Exact Rubric Answers)

Q1: Why does `LiveAuctionPanel` say `extends JPanel implements AuctionObserver`?

Answer: "Java allows single class inheritance (`extends JPanel` makes it a Swing UI component) combined with interface implementation (`implements AuctionObserver` lets it receive real-time timer, bid, and sale callbacks)."

Q2: Name the standard Java libraries used across `com.cricbid.ui` and `ReportService`.

Answer: "`javax.swing` and `java.awt` (`CardLayout`, `Graphics2D`, `JFileChooser`), `java.io.PrintWriter` for CSV file output, `java.util.function` (`Consumer`, `BiConsumer`), `java.util.stream.Collectors`, and `java.net.NetworkInterface`."

Q3: Why did you add `if (auctionClient == null)` after `SoldPopupDialog` in Review 2?

Answer: "All 3 laptops show the 3-second SOLD popup, but only Laptop 1 (the Host) has `auctionClient == null`, ensuring only the Host advances to the next player instead of 3 laptops skipping 3 players simultaneously."